Status	Finished
Started	Sunday, 11 October 2026, 3:04 PM
Completed	Sunday, 11 October 2026, 3:13 PM
Duration	9 mins 15 secs
Grade	**15.00** out of 15.00 (**100**%)

Question **1**

Correct

Mark 1.00 out of 1.00

What is the first step in binary search?

a. Compare the target element with the first element in the list
b. Compare the target element with the middle element in the list ✓
c. Sort the list
d. Divide the list into two equal parts

Question **2**

Correct

Mark 1.00 out of 1.00

In linear search, how is the element searched?

a. By sorting the list first
b. By using a hash function
c. By dividing the list into halves
d. By comparing each element in the list sequentially ✓

Question **3**

Correct

Mark 1.00 out of 1.00

Which of the following scenarios is best suited for applying binary search?

a. When the list is unsorted
b. When the list contains duplicate elements
c. When the list is very small
d. When the list is sorted ✓

Question **4**

Correct

Mark 1.00 out of 1.00

Which of the following is a type of searching method?

a. Bubble search
b. Quick search
c. Linear search ✓
d. Merge search

Question **5**

Correct

Mark 1.00 out of 1.00

Which of the following is a limitation of binary search?

a. It is slower than linear search for small lists
b. It does not work with negative numbers
c. It can only be applied to large lists
d. It requires the list to be sorted ✓

Question **6**

Correct

Mark 1.00 out of 1.00

Given an array arr = {45,77,89,90,94,99,100} and key = 99; what are the mid values(corresponding array elements) in the first and second levels of recursion?

a. 89 and 99
b. 89 and 94
c. 90 and 94
d. 90 and 99 ✓

Question **7**

Correct

Mark 1.00 out of 1.00

In _____ checks the elements of a list, one at a time, without skipping any element.

a. Linear search ✓
b. Hash search
c. Both (1) & (3)
d. Binary search

Question **8**

Correct

Mark 1.00 out of 1.00

In a linear search, how many comparisons are made in the worst-case scenario to find an element in a list of size n?

a. n/2
b. log n
c. n ✓
d. 1

Question **9**

Correct

Mark 1.00 out of 1.00

Which method of searching involves sequentially comparing each element until a match is found?

a. Linear search ✓
b. Hashing
c. Binary search
d. Jump search

Question **10**

Correct

Mark 1.00 out of 1.00

In which situation is linear search more efficient than binary search?

a. When the list is small and sorted
b. When the list is small and unsorted ✓
c. When the list is large and sorted
d. When the list is large and unsorted

Question **11**

Correct

Mark 1.00 out of 1.00

What is the time complexity of binary search in the worst case?

a. O👎
b. O(log n) ✓
c. O(n log n)
d. O(1)

Question **12**

Correct

Mark 1.00 out of 1.00

What is the best-case time complexity of linear search?

a. O(log n)
b. O👎
c. O(1) ✓
d. O(n log n)

Question **13**

Correct

Mark 1.00 out of 1.00

If a list contains 1000 elements, how many comparisons would a binary search typically make in the worst case?

a. 500
b. 10 ✓
c. 1000
d. 100

Question **14**

Correct

Mark 1.00 out of 1.00

Which of the following is a conventional searching technique?

a. Dynamic search
b. Binary search
c. Linear search ✓
d. Hashing

Question **15**

Correct

Mark 1.00 out of 1.00

Which of the following is not a limitation of binary search algorithm?

a. There must be a mechanism to access middle element directly
b. Requirement of sorted array is expensive when a lot of insertion and deletions are needed
c. Binary search algorithm is not efficient when the data elements more than 1500 ✓
d. Must use a sorted array

Status	Finished
Started	Thursday, 29 October 2026, 7:43 PM
Completed	Thursday, 29 October 2026, 8:01 PM
Duration	17 mins 51 secs
Grade	**10.00** out of 10.00 (**100**%)

Question **1**

Correct

Mark 1.00 out of 1.00

Given an array `nums` containing `n` distinct numbers in the range `[0, n]`, return *the only number in the range that is missing from the array.*

Example 1:

```
Input: nums = [3,0,1]
Output: 2
Explanation: n = 3 since there are 3 numbers, so all numbers are in the range [0,3]. 2 is the missing number in the range since it does not appear in nums.
```

Example 2:

```
Input: nums = [0,1]
Output: 2
Explanation: n = 2 since there are 2 numbers, so all numbers are in the range [0,2]. 2 is the missing number in the range since it does not appear in nums.
```

Example 3:

```
Input: nums = [9,6,4,2,3,5,7,0,1]
Output: 8
Explanation: n = 9 since there are 9 numbers, so all numbers are in the range [0,9]. 8 is the missing number in the range since it does not appear in nums.
```

For example:

Test	Result
print(missingNumber([3,0,1]))	2
print(missingNumber([0,1]))	2

Answer: (penalty regime: 0 %)

Reset answer

```
def missingNumber(nums):
    n = len(nums)
    return n * (n + 1) // 2 - sum(nums)

```

	Test	Expected	Got	
✓	print(missingNumber([3,0,1]))	2	2	✓
✓	print(missingNumber([0,1]))	2	2	✓
✓	print(missingNumber([9,6,4,2,3,5,7,0,1]))	8	8	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **2**

Correct

Mark 1.00 out of 1.00

Given two Strings s1 and s2, remove all the characters from s1 which is present in s2.

Constraints

1<= string length <= 200

Sample Input 1

experience
enc

Sample Output 1

xpri

Answer: (penalty regime: 0 %)

```
s1 = input().strip()
s2 = input().strip()

result = ""
for ch in s1:
    if ch not in s2:
        result += ch

print(result)
```

	Input	Expected	Got	
✓	experience enc	xpri	xpri	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **3**

Correct

Mark 1.00 out of 1.00

Given an array of integers `nums` which is sorted in ascending order, and an integer `target`, write a function to search `target` in `nums`. If `target` exists, then return its index. Otherwise, return `-1`.

You must write an algorithm with `O(log n)` runtime complexity.

Example 1:

```
Input: nums = [-1,0,3,5,9,12], target = 9
Output: 4
Explanation: 9 exists in nums and its index is 4
```

Example 2:

```
Input: nums = [-1,0,3,5,9,12], target = 2
Output: -1
Explanation: 2 does not exist in nums so return -1
```

Constraints:

- `1 <= nums.length <= 10^4`
- `-10^4 < nums[i], target < 10^4`
- All the integers in `nums` are **unique**.
- `nums` is sorted in ascending order.

For example:

Test	Result
`print(search([-1,0,3,5,9,12],9))`	4

Answer: (penalty regime: 0 %)

Reset answer

```
def search(nums, target):
    left, right = 0, len(nums) - 1

    while left <= right:
        mid = (left + right) // 2

        if nums[mid] == target:
            return mid
        elif nums[mid] < target:
            left = mid + 1
        else:
            right = mid - 1

    return -1

```

	Test	Expected	Got	
✓	`print(search([-1,0,3,5,9,12],9))`	4	4	✓
✓	`print(search([-1,0,3,5,9,12],2))`	-1	-1	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **4**

Correct

Mark 1.00 out of 1.00

Two string values S1, S2 are passed as the input. The program must print first N characters present in S1 which are also present in S2.

Input Format:

The first line contains S1.
The second line contains S2.
The third line contains N.

Output Format:

The first line contains the N characters present in S1 which are also present in S2.

Boundary Conditions:

2 <= N <= 10
2 <= Length of S1, S2 <= 1000

Example Input/Output 1:

Input:

abcbde
cdefghbb
3

Output:

bcd

Note:

b occurs twice in common but must be printed only once.

Answer: (penalty regime: 0 %)

```
s1 = input().strip()
s2 = input().strip()
n = int(input())

result = ""

for ch in s1:
    if ch in s2 and ch not in result:
        result += ch
    if len(result) == n:
        break

print(result)
```

	Input	Expected	Got	
✓	abcbde cdefghbb 3	bcd	bcd	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **5**

Correct

Mark 1.00 out of 1.00

An list contains N numbers and you want to determine whether two of the numbers sum to a given number K. For example, if the input is 8, 4, 1, 6 and K is 10, the answer is yes (4 and 6). A number may be used twice.

Input Format

The first line contains a single integer n , the length of list

The second line contains n space-separated integers, list[i].

The third line contains integer k.

Output Format

Print Yes or No.

Sample Input

7

0 1 2 4 6 5 3

1

Sample Output

Yes

For example:

Input	Result
5 8 9 12 15 3 11	Yes
6 2 9 21 32 43 43 1 4	No

Answer: (penalty regime: 0 %)

```
n = int(input())
nums = list(map(int, input().split()))
k = int(input())

seen = set()
found = False

for num in nums:
    if k - num in seen or (num * 2 == k and nums.count(num) >= 2):
        found = True
        break
    seen.add(num)

print("Yes" if found else "No")

```

	Input	Expected	Got	
✓	5 8 9 12 15 3 11	Yes	Yes	✓
✓	6 2 9 21 32 43 43 1 4	No	No	✓
✓	6 13 42 31 4 8 9 17	Yes	Yes	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **6**

Correct

Mark 1.00 out of 1.00

Given an list, find peak element in it. A peak element is an element that is greater than its neighbors.

An element a[i] is a peak element if

A[i-1] <= A[i] >=a[i+1] for middle elements. [0<i<n-1]

A[i-1] <= A[i] for last element [i=n-1]

A[i]>=A[i+1] for first element [i=0]

Input Format

The first line contains a single integer n , the length of A .
The second line contains n space-separated integers,A[i].

Output Format

Print peak numbers separated by space.

Sample Input

5

8 9 10 2 6

Sample Output

```
10 6
```

For example:

Input	Result
4 12 3 6 8	12 8

Answer: (penalty regime: 0 %)

```
n = int(input())
A = list(map(int, input().split()))

peaks = []

for i in range(n):
    left_ok = (i == 0 or A[i] >= A[i - 1])
    right_ok = (i == n - 1 or A[i] >= A[i + 1])

    if left_ok and right_ok:
        peaks.append(A[i])

print(*peaks)
```

	Input	Expected	Got	
✓	7 15 7 10 8 9 4 6	15 10 9 6	15 10 9 6	✓
✓	4 12 3 6 8	12 8	12 8	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **7**

Correct

Mark 1.00 out of 1.00

Balanced strings are those that have an equal quantity of 'L' and 'R' characters.

Given a balanced string s, split it in the maximum amount of balanced strings.

Return the maximum amount of split balanced strings.

Example 1:

Input:

RLRRLLRLRL

Output:

4

Explanation: s can be split into "RL", "RRLL", "RL", "RL", each substring contains same number of 'L' and 'R'.

Example 2:

Input:

RLLLLRRRLR

Output:

3

Explanation: s can be split into "RL", "LLLRRR", "LR", each substring contains same number of 'L' and 'R'.

Example 3:

Input:

LLLLRRRR

Output:

1

Explanation: s can be split into "LLLLRRRR".

Constraints:

1 <= s.length <= 1000

s[i] is either 'L' or 'R'.

s is a balanced string.

For example:

Test	Result
`print(BalancedStrings('RLRRLLRLRL'))`	4
`print(BalancedStrings('RLLLLRRRLR'))`	3

Answer: (penalty regime: 0 %)

Reset answer

```
def BalancedStrings(s):
    count = 0
    balance = 0

    for ch in s:
        if ch == 'R':
            balance += 1
        else:
            balance -= 1

        if balance == 0:
            count += 1

    return count
```

	Test	Expected	Got	
✓	print(BalancedStrings('RLRRLLRLRL'))	4	4	✓
✓	print(BalancedStrings('RLLLLRRRLR'))	3	3	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **8**

Correct

Mark 1.00 out of 1.00

String should contain only the words are not palindrome.

Sample Input 1

Malayalam is my mother tongue

Sample Output 1

is my mother tongue

Answer: (penalty regime: 0 %)

```
s = input().lower()
print(" ".join(w for w in s.split() if w!=w[::-1]))
```

	Input	Expected	Got	
✓	Malayalam is my mother tongue	is my mother tongue	is my mother tongue	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **9**

Correct

Mark 1.00 out of 1.00

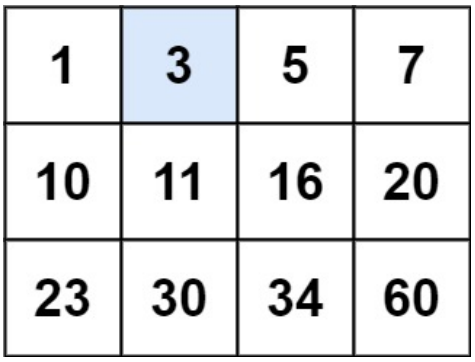
You are given an `m x n` integer matrix `matrix` with the following two properties:

- Each row is sorted in non-decreasing order.
- The first integer of each row is greater than the last integer of the previous row.

Given an integer `target`, return `True` *if* `target` *is in* `matrix` *or* `False` *otherwise*.

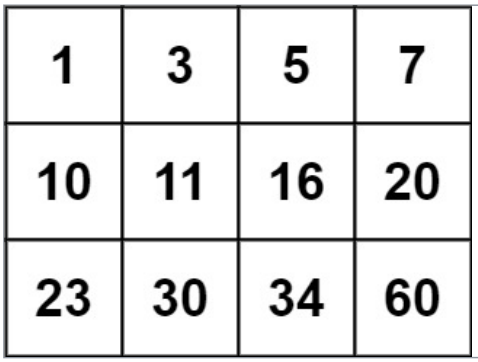
You must write a solution in `O(log(m * n))` time complexity.

Example 1:




```
Input: matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]], target = 3
Output: True
```

Example 2:




```
Input: matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]], target = 13
Output: False
```

For example:

Test	Result
print(searchMatrix([[1,3,5,7],[10,11,16,20],[23,30,34,60]], 13))	False
print(searchMatrix([[1,3,5,7],[10,11,16,20],[23,30,34,60]], 3))	True

Answer: (penalty regime: 0 %)

Reset answer

```
def searchMatrix(matrix, target):
    m, n = len(matrix), len(matrix[0])
    left, right = 0, m * n - 1

    while left <= right:
        mid = (left + right) // 2
        value = matrix[mid // n][mid % n]

        if value == target:
            return True
        elif value < target:
            left = mid + 1
        else:
```

```
14              right = mid - 1
15
16      return False
17
```

	Test	Expected	Got	
✓	print(searchMatrix([[1,3,5,7],[10,11,16,20],[23,30,34,60]], 13))	False	False	✓
✓	print(searchMatrix([[1,3,5,7],[10,11,16,20],[23,30,34,60]], 3))	True	True	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **10**

Correct

Mark 1.00 out of 1.00

Write a Python program for binary search.

For example:

Input	Result
1,2,3,5,8 6	False
3,5,9,45,42 42	True

Answer: (penalty regime: 0 %)

```
nums = list(map(int, input().split(',')))
target = int(input())

nums.sort()
left, right = 0, len(nums) - 1
found = False

while left <= right:
    mid = (left + right) // 2

    if nums[mid] == target:
        found = True
        break
    elif nums[mid] < target:
        left = mid + 1
    else:
        right = mid - 1

print(found)
```

	Input	Expected	Got	
✓	1,2,3,5,8 6	False	False	✓
✓	3,5,9,45,42 42	True	True	✓
✓	52,45,89,43,11 11	True	True	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.